

Mastering Azure DevOps Pipelines for Absolute Beginners

Azure DevOps pipelines save you hours of manual work by automating how your code travels from your laptop to the cloud. Learning this skill early makes you instantly valuable to any modern engineering team.

What you'll learn:

1. Creating Your First Pipeline File
2. Installing Dependencies Automatically
3. Deploying Code to Azure Web Apps
4. Securing Secrets in Pipeline Variables

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

1. Creating Your First Pipeline File

Every pipeline starts with a configuration file named `azure-pipelines.yml` placed in your code repository. This file acts as a blueprint telling Azure what steps to take. You can write it in plain text using YAML syntax. It keeps your deployment process repeatable and transparent for your entire team.

```
trigger:
  - main

pool:
  vmImage: 'ubuntu-latest'

steps:
  - script: echo 'Hello, Azure DevOps!'
    displayName: 'Run a simple script'
```

Cloud & DevOps Daily

Azure DevOps Pipelines: Ship Code to Azure in Minutes

Friday, October 02, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

2. Installing Dependencies Automatically

Before building your app, your pipeline needs all the required packages and libraries installed. Instead of doing this manually, you add a step in your pipeline to fetch them. This ensures your app builds in a clean, fresh environment every single time without missing files.

```
$steps:  
- script: |  
    npm install  
    npm run build  
  displayName: 'Install and Build App'
```

Cloud & DevOps Daily

Azure DevOps Pipelines: Ship Code to Azure in Minutes

Friday, October 02, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

3. Deploying Code to Azure Web Apps

Once your code is built and tested, it is ready to go live. You can use official Azure tasks in your pipeline to publish your artifacts directly to an Azure App Service. This replaces the old way of FTP uploading and reduces human error to zero.

```
$- task: AzureWebApp@1
  inputs:
    azureSubscription: 'Your-Service-Connection-Name'
    appName: 'your-unique-app-name'
    package: '$(System.DefaultWorkingDirectory)/**/*.zip'
```

Cloud & DevOps Daily

Azure DevOps Pipelines: Ship Code to Azure in Minutes

Friday, October 02, 2026

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

4. Securing Secrets in Pipeline Variables

Never put API keys or passwords directly in your pipeline configuration file. Instead, use Azure DevOps Pipeline Variables and lock them as secrets. The pipeline will securely inject them during runtime without exposing them in your logs.

```
$variables:  
  - name: dbPassword  
    value: $[variables.MySecretPasswordVariable]
```

Cloud & DevOps Daily

Azure DevOps Pipelines: Ship Code to Azure in Minutes

Friday, October 02, 2026

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

- + Keep your pipeline files small and easy to read
- + Use descriptive display names for every step
- + Test changes on a separate branch before pushing to main
- + Use YAML lint tools to catch indentation errors early
- + Clean up temporary build artifacts to save storage

Watch Out For

- ! Hardcoding passwords or tokens directly into the YAML file
- ! Using wrong indentation which causes pipeline syntax errors
- ! Forgetting to link your Azure subscription to your DevOps project

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mrmanrajsingh